

Chapter 4: System Implementation

4.2 Mobile Application (Flutter Framework) Implementation

4.2.1 Introduction

This section presents the implementation of the **Mobile Application track** for the Smart Guide graduation project. The mobile application serves as the primary on-the-go client for tourists, providing a responsive and localized user interface, structured navigation, secure session handling, and seamless integration with the backend services.

The implementation emphasizes:

- A scalable **feature-based project structure**
- A modern navigation system using **GoRouter**
- Predictable state handling using **BLoC/Cubit**
- Consistent UI scaling using **ScreenUtil**
- Multi-language support (Arabic/English) with correct **RTL/LTR** behavior

4.2.2 Tools and Languages

The Mobile Application module was developed using the following technologies and tools:

4.2.2.1 Flutter and Dart

- **Flutter Framework**: Used to build the mobile user interface and deliver a cross-platform experience.
- **Dart Language**: Used as the main programming language for UI, state, and integration logic.

4.2.2.2 Navigation and Routing

- **GoRouter (`go_router`)**: Used for declarative routing, named routes, path parameters, and passing objects using `extra`.

4.2.2.3 State Management

- **BLoC/Cubit (`flutter_bloc`)**: Used to separate presentation from business logic and produce stable, testable UI states.

4.2.2.4 Responsive UI

- **ScreenUtil (`flutter_screenutil`)**: Used to scale paddings, font sizes, and component dimensions consistently across devices.

4.2.3 Core Feature Implementation

4.2.3.1 AI Guide Interface using Sliver Architecture (CustomScrollView/SliverAppBar)

The AI Guide screen is implemented using a **Sliver-based architecture** to achieve a modern, smooth, and scalable UI layout. The screen is built using `CustomScrollView` and a pinned `SliverAppBar` (encapsulated in a reusable widget), enabling:

- A consistent header area that remains visible during scrolling (`pinned: true`)
- Flexible expansion behavior (`expandedHeight`)
- Efficient list rendering through `SliverList` delegation
- A clean separation between the app bar sliver and the dynamic content sliver

The header is implemented through `CustomSliverAppBarWidget`, which wraps a `SliverAppBar` to ensure consistent styling across different screens.

4.2.3.2 Dynamic Chat Logic (Transitioning from Empty State to Active Chat)

The chat experience is designed to transition between two primary UI states:

1. **Empty State:** Displayed before the user sends any message. It is implemented as a sliver (`EmptyChatSliver`) that provides guidance text and an illustration to explain the feature.
2. **Active Chat State:** Triggered after the user sends the first message. The screen switches to a scrollable chat list sliver (`SliverList`) and enables scroll physics.

This behavior is controlled by a boolean flag `isChatStarted`, which changes when the user presses the send action. The scroll physics also adapt accordingly:

- When chat is not started, scrolling is disabled to keep the focus on the onboarding/empty UI.
- When chat is started, bouncing scroll physics are enabled for a natural chat browsing experience.

4.2.4 Main Codes

This section presents the key code snippets that demonstrate the Sliver-based AI Guide layout and the auto-expanding chat input component.

4.2.4.1 Sliver-based AI Guide Screen Implementation

```
class AiGuideScreen extends StatefulWidget {
  const AiGuideScreen({super.key});

  @override
  State<AiGuideScreen> createState() => _AiGuideScreenState();
}

class _AiGuideScreenState extends State<AiGuideScreen> {
  bool isChatStarted = false;
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    resizeToAvoidBottomInset: true,
    backgroundColor: AppColors.backgroundColor,
    body: SafeArea(
      child: Stack(
        children: [
          CustomScrollView(
            physics: isChatStarted
              ? const BouncingScrollPhysics()
              : const NeverScrollableScrollPhysics(),
            slivers: [
              CustomSliverAppBarWidget(
                title: LocaleKeys.aiTravelCompanion.tr(),
              ),
              isChatStarted ? buildChatList() : EmptyChatSliver(),
            ],
          ),
          Align(
            alignment: Alignment.bottomCenter,
            child: ChatInputWidget(
              onSend: () {
                setState(() {
                  isChatStarted = true;
                });
              },
            ),
          ),
        ],
      ),
    );
}

Widget buildChatList() {
  return SliverPadding(
    padding: EdgeInsets.fromLTRB(16.w, 8.h, 16.w, 100.h),
    sliver: SliverList(
      delegate: SliverChildBuilderDelegate((context, index) {
        return const Text("Chat Message Holder...");
      }, childCount: 10),
    ),
  );
}

```

```
);  
}  
}
```

4.2.4.2 Auto-expanding Chat Input Widget Implementation

The chat input is implemented using a multiline `TextFormField` configured with:

- `maxLines: null` to allow vertical expansion
- `minLines: 1` to preserve a compact initial height
- A fixed bottom container aligned to the screen to remain accessible while scrolling

```
class ChatInputWidget extends StatelessWidget {  
  final VoidCallback onSend;  
  const ChatInputWidget({super.key, required this.onSend});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      padding: EdgeInsets.fromLTRB(16.w, 10.h, 16.w, 20.h),  
      color: AppColors.backgroundColor,  
      child: Container(  
        padding: EdgeInsets.symmetric(horizontal: 16.w, vertical: 4.h),  
        decoration: BoxDecoration(  
          color: Colors.white,  
          borderRadius: BorderRadius.circular(12.r),  
          boxShadow: [  
            BoxShadow(  
              color: Colors.black.withOpacity(0.05),  
              blurRadius: 10,  
              offset: const Offset(0, -2),  
            ),  
          ],  
        ),  
        child: Row(  
          crossAxisAlignment: CrossAxisAlignment.end,  
          children: [  
            Expanded(  
              child: Theme(  
                data: ThemeData(  
                  textSelectionTheme: TextSelectionThemeData(  
                    selectionColor: AppColors.primaryColor.withOpacity(0.5),  
                    selectionHandleColor: AppColors.primaryColor,  
                  ),  
                ),  
              ),  
            ],  
          ),  
        ),  
      ),  
    );  
  }  
}
```

```

child: TextFormField(
  cursorColor: AppColors.primaryColor,
  maxLines: null,
  minLines: 1,
  keyboardType: TextInputType.multiline,
  decoration: InputDecoration(
    hintText: LocaleKeys.aiGuidePlaceholder.tr(),
    border: InputBorder.none,
    contentPadding: EdgeInsets.symmetric(vertical: 12.h),
  ),
),
),
),
),
Padding(
  padding: EdgeInsets.only(bottom: 12.h),
  child: Row(
    children: [
      Icon(
        Icons.keyboard_arrow_down_sharp,
        color: Colors.grey[600],
      ),
      CustomWidthSpacingWidget(width: 10.w),
      GestureDetector(
        onTap: onSend,
        child: SvgPicture.asset(Assets.imagesSvgArrowSend),
      ),
    ],
  ),
),
];
);
}
}

```

4.2.5 Testing and Evaluation

The testing and evaluation of the mobile application focused on practical usability, UI responsiveness, and localization correctness across different devices.

4.2.5.1 UI/UX Responsiveness Testing

The application UI was validated to ensure consistent scaling across multiple screen sizes using:

- `ScreenUtilInit` with a fixed reference design size
- Responsive dimension usage (`w`, `h`, `sp`, `r`) throughout the UI

Key evaluation points:

- Chat input expansion does not break layout or overlap essential UI elements.
- Sliver-based layout maintains stable scrolling behavior under different content lengths.
- Bottom-aligned chat input remains accessible while content scrolls.

4.2.5.2 Localization and RTL/LTR Testing

The application supports Arabic and English using `easy\_localization`. Testing included:

- Verifying correct text rendering and translation lookup using generated localization keys.
- Confirming RTL/LTR behavior:
- Proper text alignment and hint direction
- Stable input field behavior in Arabic (RTL) and English (LTR)
- Ensuring that UI spacing and icon direction do not degrade in RTL layout.

4.2.6 Future Work

Based on the current implementation, several enhancements can significantly improve the mobile module:

4.2.6.1 AI Voice Integration

- Add speech-to-text for user prompts and text-to-speech for AI responses, enabling hands-free interaction during tours.
- Support multi-language voice output and configurable speaking speed.

4.2.6.2 Advanced AR Navigation

- Extend the exploration experience by integrating AR overlays for directions, landmark highlighting, and contextual information.
- Provide route-aware guidance that adapts in real time based on user location and movement.